

На этом семинаре мы:

- Узнаем о том, что такое переобучение, что такое линейная регрессия и как не допустить переобучения в линейных моделях;
- Обучим линейную регрессию для предсказания цены дома;
- Научимся работать с разными типами признаков;
- Поймем, в чем отличие между разными регуляризаторами;
- Научимся пользоваться основными инструментами в `sklearn`: моделями, трансформерами и `pipeline`;
- Обсудим преобразования признаков и целевой переменной, которые могут помочь в обучении линейных моделей.

```
In [1]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
import warnings
warnings.simplefilter("ignore")
%matplotlib inline

sns.set(style="darkgrid")
```

Часть 0. Введение в линейные модели

Напомним, что линейная регрессия — это модель следующего вида:

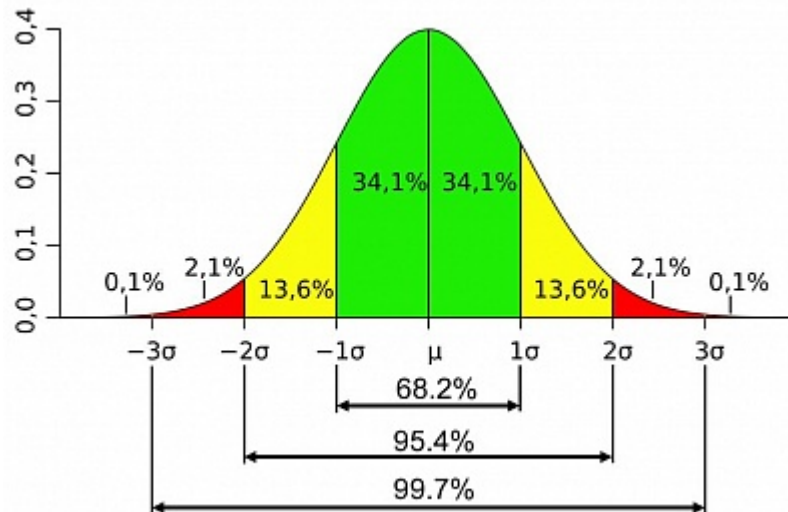
$$a(x) = \langle w, x \rangle + w_0$$

где $w \in \mathbb{R}^d$, $w_0 \in \mathbb{R}$. Обучить линейную регрессию — значит найти w и w_0 .

В машинном обучении часто говорят об *обобщающей способности модели*, то есть о способности модели работать на новых тестовых данных хорошо. Если модель будет идеально предсказывать выборку, на которой она обучалась, но при этом просто ее запомнит, не "вытащив" из данных никакой закономерности, от нее будет мало толку. Такую модель называют *переобученной*: она слишком подстроилась под обучающие примеры, не выявив никакой полезной закономерности, которая позволила бы ей совершать хорошие предсказания на данных, которые она ранее не видела.

Рассмотрим следующий пример, на котором будет хорошо видно, что значит переобучение модели. Для этого нам понадобится сгенерировать синтетические данные. Рассмотрим зависимость $y(x) = \cos(1.5\pi x)$, y — целевая переменная, а x — объект (просто число от 0 до 1). В жизни мы наблюдаем какое-то конечное количество пар объект-таргет, поэтому смоделируем это, взяв 30 случайных точек x_i в отрезке $[0; 1]$. Более того, в реальной жизни целевая переменная может быть зашумленной (измерения в жизни не всегда точны), смоделируем это, зашумив значение функции нормальным шумом: $\tilde{y}_i = y(x_i) + \mathcal{N}(0, 0.01)$.

$\mathcal{N}(\mu, \sigma^2)$ – это нормальное/гауссовское распределение, график такого распределения приведен ниже.



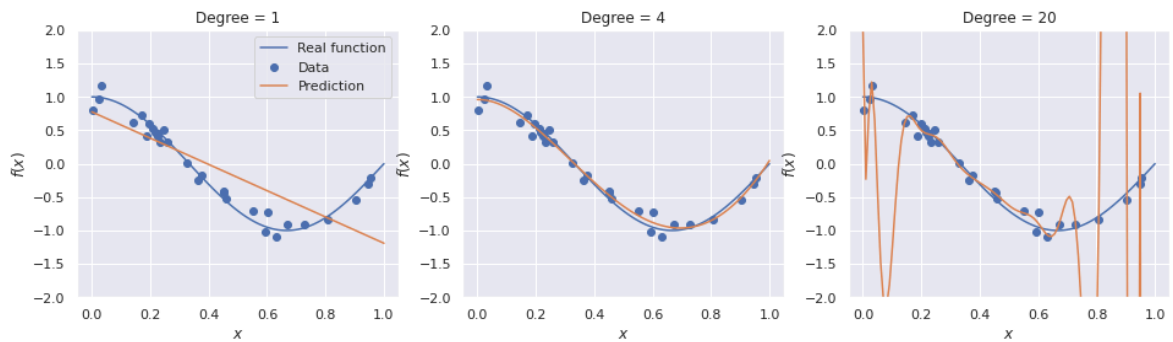
Попытаемся обучить три разных линейных модели: признаки для первой — $\{x\}$, для второй — $\{x, x^2, x^3, x^4\}$, для третьей — $\{x, \dots, x^{20}\}$.

```
In [2]: from sklearn.linear_model import LinearRegression

np.random.seed(36)
x = np.linspace(0, 1, 100)
y = np.cos(1.5 * np.pi * x)

x_objects = np.random.uniform(0, 1, size=30)
y_objects = np.cos(1.5 * np.pi * x_objects) + np.random.normal(scale=0.1,

from sklearn.preprocessing import PolynomialFeatures
fig, axs = plt.subplots(figsize=(16, 4), ncols=3)
for i, degree in enumerate([1, 4, 20]):
    X_objects = PolynomialFeatures(degree, include_bias=False).fit_transform(x_objects)
    X = PolynomialFeatures(degree, include_bias=False).fit_transform(x[:, :30])
    regr = LinearRegression().fit(X_objects, y_objects)
    y_pred = regr.predict(X)
    axs[i].plot(x, y, label="Real function")
    axs[i].scatter(x_objects, y_objects, label="Data")
    axs[i].plot(x, y_pred, label="Prediction")
    if i == 0:
        axs[i].legend()
    axs[i].set_title("Degree = %d" % degree)
    axs[i].set_xlabel("$x$")
    axs[i].set_ylabel("$f(x)$")
    axs[i].set_ylim(-2, 2)
```



Вопрос 1: Почему первая модель получилась плохой, а третья переобучилась?

Чтобы избежать переобучения, модель регуляризуют. Обычно переобучения в линейных моделях связаны с большими весами, а поэтому модель часто штрафуют за большие значения весов, добавляя к функционалу качества, например, квадрат ℓ^2 -нормы вектора w :

$$Q_{reg}(X, y, a) = Q(X, y, a) + \lambda \|w\|_2^2$$

Это слагаемое называют ℓ_2 -регуляризатором, а коэффициент λ — коэффициентом регуляризации. Более подробно о регуляризации и переобучении вы узнаете из лекций.

Вопрос 2: Почему большие веса в линейной модели — плохо?

Вопрос 3: Почему регуляризовать w_0 — плохая идея?

Вопрос 4: На что влияет коэффициент λ ? Что будет происходить с моделью, если λ начать уменьшать? Что будет, если λ сделать слишком большим?

Часть 1. Загружаем данные

Мы будем работать с данными из соревнования [House Prices: Advanced Regression Techniques](#), в котором требовалось предсказать стоимость жилья.

Давайте сначала загрузим и немного изучим данные (можно взять `train.csv` со страницы соревнования и переименовать в `train_sem2.csv`).

```
In [3]: !curl -L -o 'train_sem2.csv' 'https://www.dropbox.com/s/6dxq90t0prn2vaw/_
```

```
In [4]: data = pd.read_csv("train_sem2.csv")
data.head()
```

```
Out [4]:
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	Lan
0	1	60	RL	65.0	8450	Pave	NaN	Reg	
1	2	20	RL	80.0	9600	Pave	NaN	Reg	
2	3	60	RL	68.0	11250	Pave	NaN	IR1	
3	4	70	RL	60.0	9550	Pave	NaN	IR1	
4	5	60	RL	84.0	14260	Pave	NaN	IR1	

5 rows x 81 columns

```
In [5]: data.shape
```

```
Out[5]: (1460, 81)
```

```
In [6]: data.columns
```

```
Out[6]: Index(['Id', 'MSSubClass', 'MSZoning', 'LotFrontage', 'LotArea', 'Street',
              'Alley', 'LotShape', 'LandContour', 'Utilities', 'LotConfig',
              'LandSlope', 'Neighborhood', 'Condition1', 'Condition2', 'BldgType',
              'HouseStyle', 'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd',
              'RoofStyle', 'RoofMatl', 'Exterior1st', 'Exterior2nd', 'MasVnrType',
              'MasVnrArea', 'ExterQual', 'ExterCond', 'Foundation', 'BsmtQual',
              'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinSF1',
              'BsmtFinType2', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', 'Heating',
              'HeatingQC', 'CentralAir', 'Electrical', '1stFlrSF', '2ndFlrSF',
              'LowQualFinSF', 'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath', 'FullBath',
              'HalfBath', 'BedroomAbvGr', 'KitchenAbvGr', 'KitchenQual',
              'TotRmsAbvGrd', 'Functional', 'Fireplaces', 'FireplaceQu', 'GarageType',
              'GarageYrBlt', 'GarageFinish', 'GarageCars', 'GarageArea', 'GarageQual',
              'GarageCond', 'PavedDrive', 'WoodDeckSF', 'OpenPorchSF',
              'EnclosedPorch', '3SsnPorch', 'ScreenPorch', 'PoolArea', 'PoolQC',
              'Fence', 'MiscFeature', 'MiscVal', 'MoSold', 'YrSold', 'SaleType',
              'SaleCondition', 'SalePrice'],
              dtype='object')
```

Первое, что стоит заметить — у нас в данных есть уникальное для каждого объекта поле `id`. Обычно такие поля только мешают и способствуют переобучению. Удалим это поле из данных.

Разделим данные на обучающую и тестовую выборки. Для простоты не будем выделять дополнительно валидационную выборку (хотя это обычно стоит делать, она нужна для подбора гиперпараметров модели, то есть параметров, которые нельзя подбирать по обучающей выборке). Дополнительно нам придется отделить значения целевой переменной от данных.

Вопрос 1: Почему поля типа id могут вызвать переобучение модели (не обязательно линейной)?

Вопрос 2: Почему стоит дополнительно отделять валидационную выборку?

Вопрос 3: Обратите внимание на фиксацию `random_state` при сплите данных. Почему это важно?

```
In [7]: from sklearn.model_selection import train_test_split

data = data.drop(columns=["Id"])

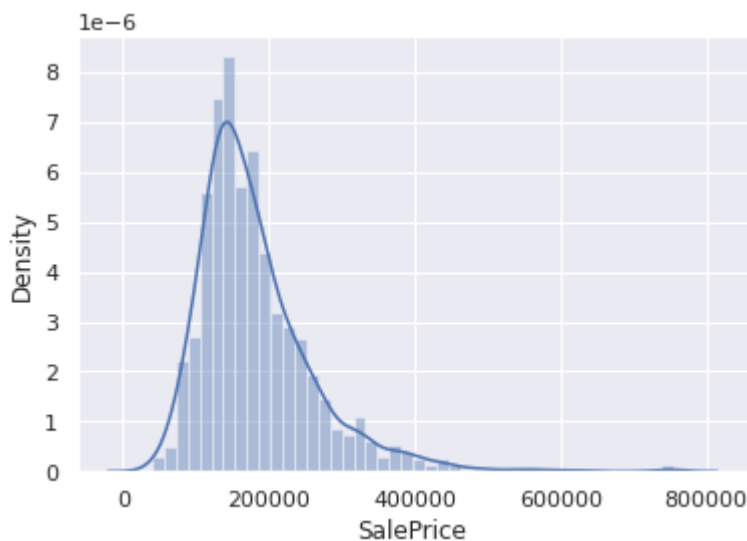
y = data["SalePrice"]
X = data.drop(columns=["SalePrice"])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
```

Посмотрим сначала на значения целевой переменной.

```
In [8]: sns.distplot(y_train)
```

```
Out[8]: <AxesSubplot:xlabel='SalePrice', ylabel='Density'>
```



Судя по гистограмме, у нас есть примеры с нетипично большой стоимостью, что может помешать нам, если наша функция потерь слишком чувствительна к выбросам. В дальнейшем мы рассмотрим способы, как минимизировать ущерб от этого.

Так как для решения нашей задачи мы бы хотели обучить линейную регрессию, было бы хорошо найти признаки, "наиболее линейно" связанные с целевой переменной, иначе говоря, посмотреть на коэффициент корреляции Пирсона между признаками и целевой переменной. Корреляция Пирсона записывается следующим образом:

$$\text{corr}(X, Y) = \frac{\text{cov}_{XY}}{\sigma_X \sigma_Y} = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^n (X_i - \bar{X})^2 \sum_{i=1}^n (Y_i - \bar{Y})^2}}$$

Вопрос: Что означает, что коэффициент корреляции Пирсона между двумя случайными величинами равен 1? -1? 0?

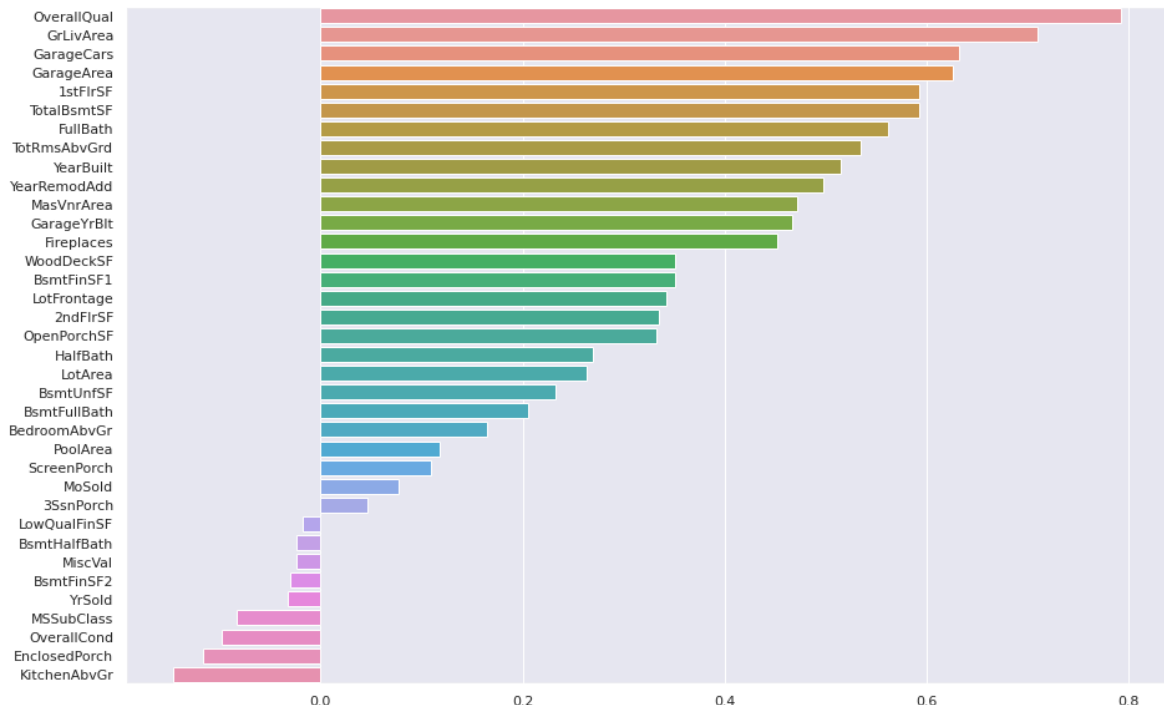
Заметим, что не все признаки являются числовыми, пока что мы не будем рассматривать такие признаки.

```
In [9]: numeric_data = X_train.select_dtypes([np.number])
numeric_data_mean = numeric_data.mean()
numeric_features = numeric_data.columns

X_train = X_train.fillna(numeric_data_mean)
X_test = X_test.fillna(numeric_data_mean)

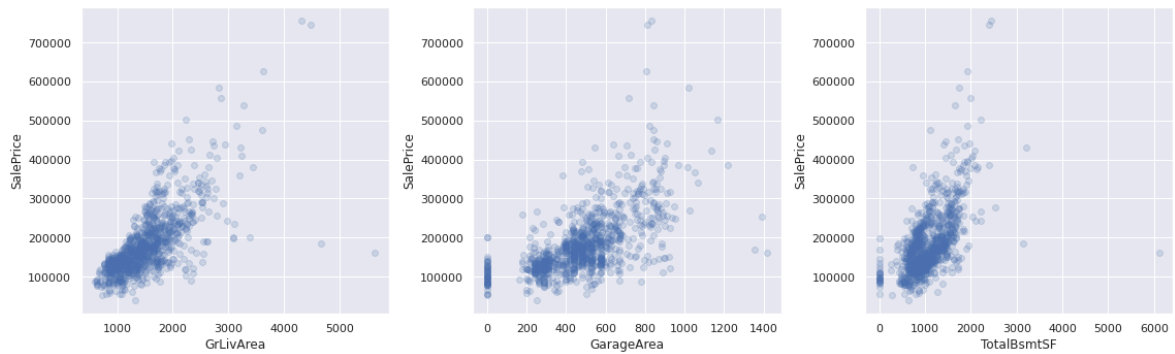
correlations = X_train[numeric_features].corrwith(y_train).sort_values(ascending=False)

plot = sns.barplot(y=correlations.index, x=correlations)
plot.figure.set_size_inches(15, 10)
```



Посмотрим на признаки из начала списка. Для этого нарисуем график зависимости целевой переменной от каждого из признаков. На этом графике каждая точка соответствует паре признак-таргет (такие графики называются scatter-plot).

```
In [10]: fig, axs = plt.subplots(figsize=(16, 5), ncols=3)
for i, feature in enumerate(["GrLivArea", "GarageArea", "TotalBsmtSF"]):
    axs[i].scatter(X_train[feature], y_train, alpha=0.2)
    axs[i].set_xlabel(feature)
    axs[i].set_ylabel("SalePrice")
plt.tight_layout()
```



Видим, что между этими признаками и целевой переменной действительно наблюдается линейная зависимость.

Часть 2. Первая модель

Немного об обучении моделей. В арсенале дата-саентиста кроме `pandas` и `matplotlib` должны быть библиотеки, позволяющие обучать модели. Для простых моделей (линейные модели, решающее дерево, ...) отлично подходит `sklearn`: в нем очень понятный и простой интерфейс. Несмотря на то, что в `sklearn` есть реализация бустинга и простых нейронных сетей, ими все же не пользуются и предпочитают специализированные библиотеки: `XGBoost`, `LightGBM` и пр. для градиентного бустинга над деревьями, `PyTorch`, `Tensorflow` и пр. для нейронных сетей. Так как мы будем обучать линейную регрессию, нам подойдет реализация из `sklearn`.

Попробуем обучить линейную регрессию на числовых признаках из нашего датасета. В `sklearn` есть несколько классов, реализующих линейную регрессию:

- `LinearRegression` — "классическая" линейная регрессия с оптимизацией MSE. Веса находятся как точное решение:

$$w^* = (X^T X)^{-1} X^T y$$
- `Ridge` — линейная регрессия с оптимизацией MSE и ℓ_2 -регуляризацией
- `Lasso` — линейная регрессия с оптимизацией MSE и ℓ_1 -регуляризацией

У моделей из `sklearn` есть методы `fit` и `predict`. Первый принимает на вход обучающую выборку и вектор целевых переменных и обучает модель, второй, будучи вызванным после обучения модели, возвращает предсказание на выборке. Попробуем обучить нашу первую модель на числовых признаках, которые у нас сейчас есть:

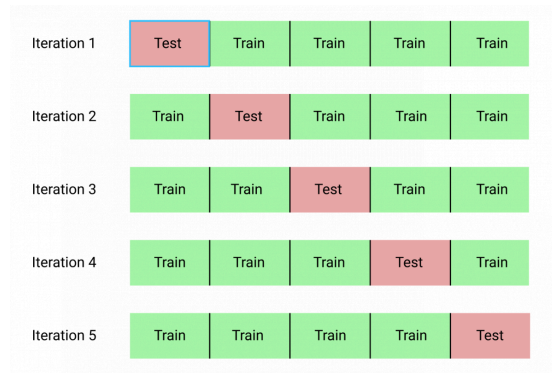
```
In [11]: from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error

model = Ridge()
model.fit(X_train[numeric_features], y_train)
y_pred = model.predict(X_test[numeric_features])
y_train_pred = model.predict(X_train[numeric_features])
```

```
print("Test RMSE = %.4f" % mean_squared_error(y_test, y_pred, squared=False))
print("Train RMSE = %.4f" % mean_squared_error(y_train, y_train_pred, squared=False))
```

Test RMSE = 32085.7681
Train RMSE = 35631.3131

Мы обучили первую модель и даже посчитали ее качество на отложенной выборке! Давайте теперь посмотрим на то, как можно оценить качество модели с помощью кросс-валидации. Принцип кросс-валидации изображен на рисунке



При кросс-валидации мы делим обучающую выборку на n частей (fold). Затем мы обучаем n моделей: каждая модель обучается при отсутствии соответствующего фолда, то есть i -ая модель обучается на всей обучающей выборке, кроме объектов, которые попали в i -ый фолд (out-of-fold). Затем мы измеряем качество i -ой модели на i -ом фолде. Так как он не участвовал в обучении этой модели, мы получим "честный результат". После этого, для получения финального значения метрики качества, мы можем усреднить полученные нами n значений.

```
In [12]: from sklearn.model_selection import cross_val_score

cv_scores = cross_val_score(model, X_train[numeric_features], y_train, cv=5)
print("Cross validation scores:\n\t", "\n\t".join("%.4f" % x for x in cv_scores))
print("Mean CV MSE = %.4f" % np.mean(-cv_scores))
```

Cross validation scores:
 -38825.3172
 -45278.9230
 -31405.3380
 -33890.1048
 -73848.3061
 -24142.7915
 -45523.7066
 -52861.2908
 -26094.3376
 -29693.5005
 Mean CV MSE = 40156.3616

Обратите внимание на то, что результаты `cv_scores` получились отрицательными. Это соглашение в `sklearn` (скоринговую функцию нужно максимизировать). Поэтому все стандартные скореры называются `neg_*`, например, `neg_root_mean_squared_error`.

Обратите внимание, что по отложенной выборке и при кросс-валидации мы считаем RMSE (Root Mean Squared Error), хотя в функционале ошибки при обучении модели используется MSE.

$$\text{RMSE}(X, y, a) = \sqrt{\frac{1}{\ell} \sum_{i=1}^{\ell} (y_i - a(x_i))^2}$$

Вопрос: Почему оптимизация RMSE эквивалентна оптимизации MSE?

Если бы RMSE не входил в стандартные метрики `sklearn`, можно было бы его определить через функцию `sklearn.metrics.make_scorer`.

```
In [13]: from sklearn.metrics import make_scorer
```

```
def rmse(y_true, y_pred):
    error = (y_true - y_pred) ** 2
    return np.sqrt(np.mean(error))

rmse_scorer = make_scorer(
    rmse,
    greater_is_better=False
)
```

```
In [14]: from sklearn.model_selection import cross_val_score
```

```
cv_scores = cross_val_score(model, X_train[numeric_features], y_train, cv=5)
print("Cross validation scores:\n\t", "\n\t".join("%.4f" % x for x in cv_scores))
print("Mean CV RMSE = %.4f" % np.mean(-cv_scores))
```

Cross validation scores:

```
-38825.3172
-45278.9230
-31405.3380
-33890.1048
-73848.3061
-24142.7915
-45523.7066
-52861.2908
-26094.3376
-29693.5005
```

Mean CV RMSE = 40156.3616

Для того, чтобы иметь некоторую точку отсчета, удобно посчитать оптимальное значение функции потерь при константном предсказании.

Вопрос: Чему равна оптимальная константа для RMSE?

```
In [15]: best_constant = y_train.mean()
print("Test RMSE with best constant = %.4f" % mean_squared_error(y_test, best_constant))
print("Train RMSE with best constant = %.4f" % mean_squared_error(y_train, best_constant))
```

Test RMSE with best constant = 76636.5188

Train RMSE with best constant = 80588.6191

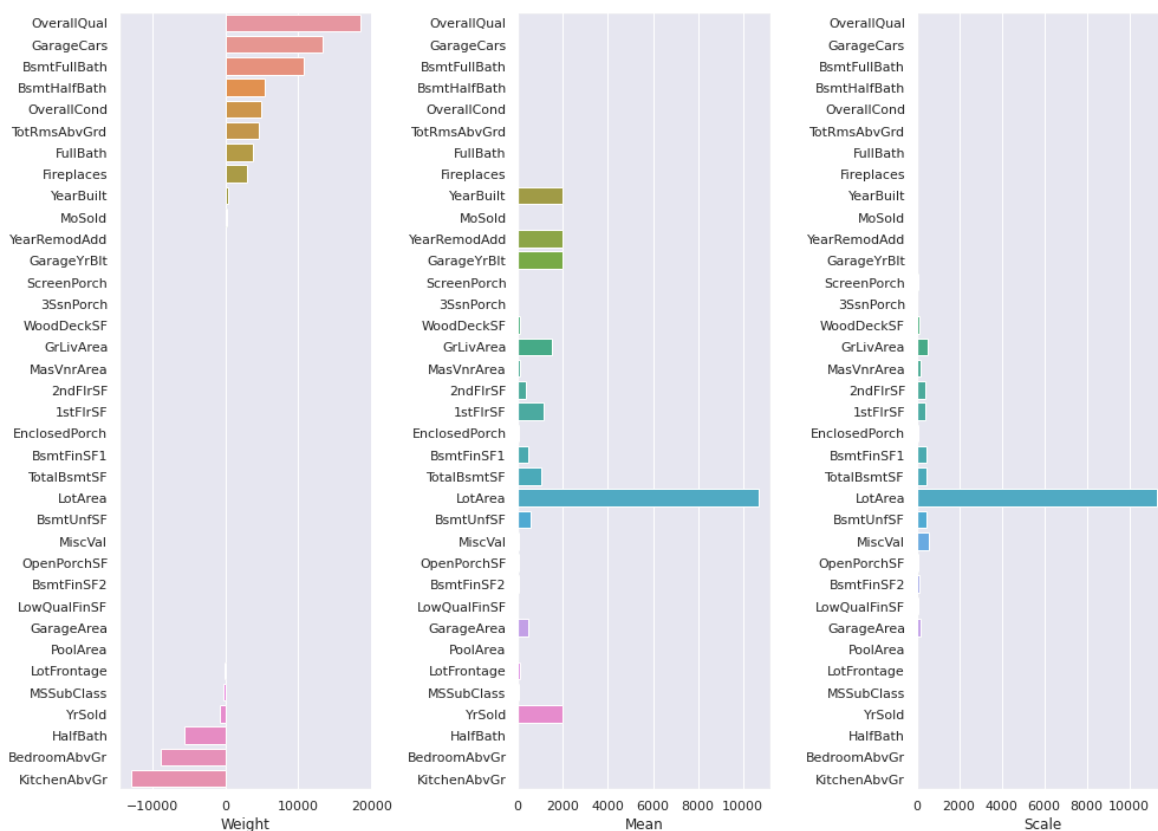
Давайте посмотрим на то, какие же признаки оказались самыми "сильными". Для этого визуализируем веса, соответствующие признакам. Чем больше вес — тем более сильным является признак.

Вопрос: Почему это не совсем правда?

```
In [16]: def show_weights(features, weights, means, scales):
    fig, axs = plt.subplots(figsize=(14, 10), ncols=3)
    sorted_weights = sorted(zip(weights, features, means, scales), reverse=True)
    weights, features, means, scales = zip(*sorted_weights)
    features = list(features)

    sns.barplot(y=features, x=list(weights), ax=axs[0])
    axs[0].set_xlabel("Weight")
    sns.barplot(y=features, x=list(means), ax=axs[1])
    axs[1].set_xlabel("Mean")
    sns.barplot(y=features, x=list(scales), ax=axs[2])
    axs[2].set_xlabel("Scale")
    plt.tight_layout()
```

```
In [17]: show_weights(numeric_features, model.coef_, X_train[numeric_features].mean())
```



Будем масштабировать наши признаки перед обучением модели. Это, среди прочего, сделает нашу регуляризацию более честной: теперь все признаки будут регуляризоваться в равной степени.

Для этого воспользуемся трансформером `StandardScaler`. Трансформеры в `sklearn` имеют методы `fit` и `transform` (а еще `fit_transform`). Метод `fit` принимает на вход обучающую выборку и считает по ней необходимые значения (например статистики, как `StandardScaler`: среднее и стандартное

отклонение каждого из признаков). `transform` применяет преобразование к переданной выборке.

```
In [18]: from sklearn.preprocessing import StandardScaler

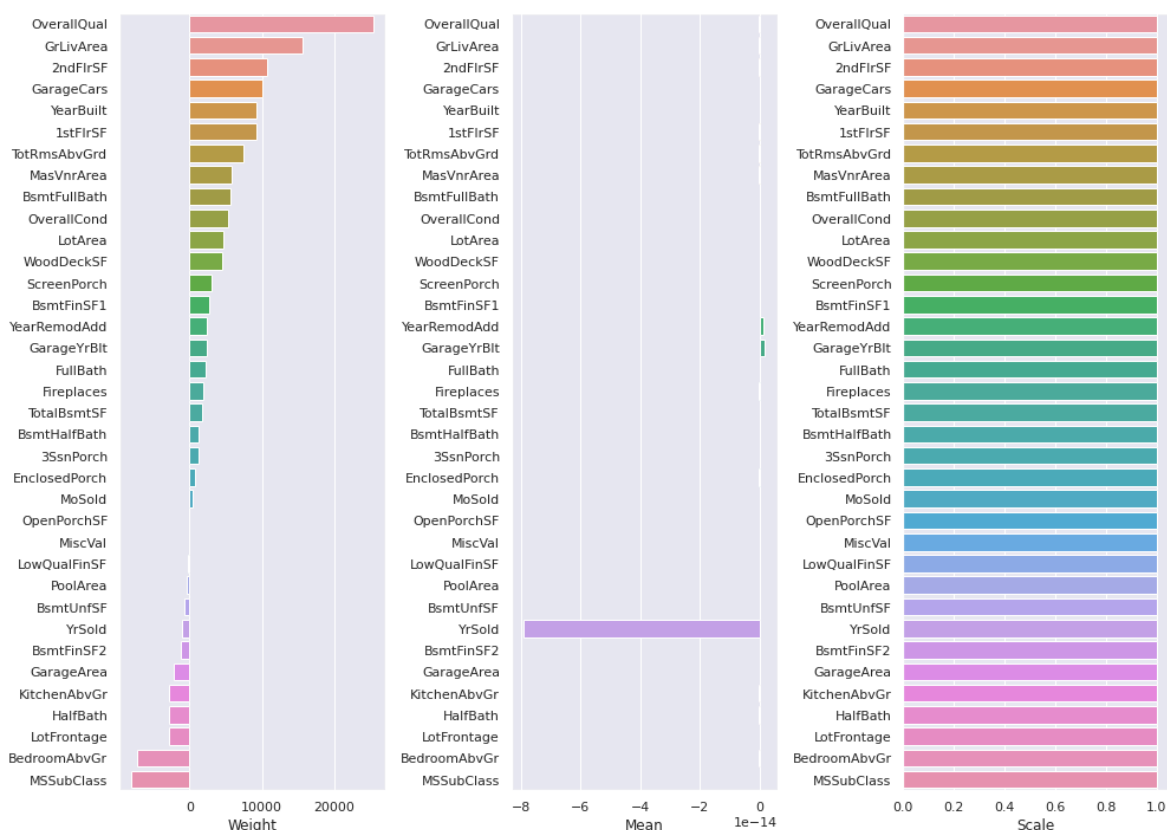
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train[numeric_features])
X_test_scaled = scaler.transform(X_test[numeric_features])

model = Ridge()
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
y_train_pred = model.predict(X_train_scaled)

print("Test RMSE = %.4f" % mean_squared_error(y_test, y_pred, squared=False))
print("Train RMSE = %.4f" % mean_squared_error(y_train, y_train_pred, squared=False))
```

Test RMSE = 32085.3150
Train RMSE = 35631.2160

```
In [19]: means = pd.Series(data=X_train_scaled.mean(axis=0), index=numeric_features)
scales = pd.Series(data=X_train_scaled.std(axis=0), index=numeric_features)
show_weights(numeric_features, model.coef_, means, scales)
```



Наряду с параметрами (веса w , w_0), которые модель оптимизирует на этапе обучения, у модели есть и гиперпараметры. У нашей модели это `alpha` — коэффициент регуляризации. Подбирают его обычно по сетке, измеряя качество на валидационной (не тестовой) выборке или с помощью кросс-валидации. Посмотрим, как это можно сделать (заметьте, что мы перебираем `alpha` по логарифмической сетке, чтобы узнать оптимальный порядок величины).

```
In [20]: from sklearn.model_selection import GridSearchCV

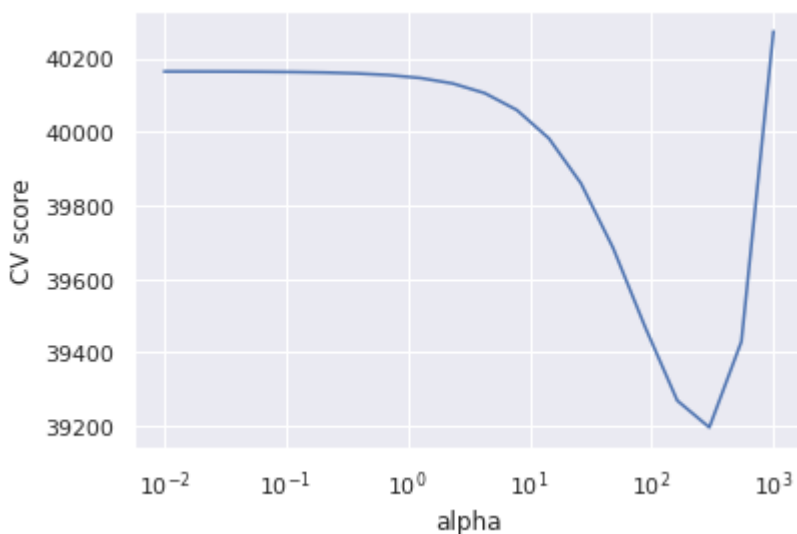
alphas = np.logspace(-2, 3, 20)
searcher = GridSearchCV(Ridge(), [{"alpha": alphas}], scoring="neg_root_m
searcher.fit(X_train_scaled, y_train)

best_alpha = searcher.best_params_["alpha"]
print("Best alpha = %.4f" % best_alpha)

plt.plot(alphas, -searcher.cv_results_["mean_test_score"])
plt.xscale("log")
plt.xlabel("alpha")
plt.ylabel("CV score")
```

Best alpha = 297.6351

Out[20]: Text(0, 0.5, 'CV score')



Вопрос: Почему мы не подбираем коэффициент регуляризации по обучающей выборке? По тестовой выборке?

Попробуем обучить модель с подобранным коэффициентом регуляризации. Заодно воспользуемся очень удобным классом `Pipeline`: обучение модели часто представляется как последовательность некоторых действий с обучающей и тестовой выборками (например, сначала нужно отмасштабировать выборку (причем для обучающей выборки нужно применить метод `fit`, а для тестовой — `transform`), а затем обучить/применить модель (для обучающей `fit`, а для тестовой — `predict`). `Pipeline` позволяет хранить эту последовательность шагов и корректно обрабатывает разные типы выборок: и обучающую, и тестовую.

```
In [21]: from sklearn.pipeline import Pipeline

simple_pipeline = Pipeline([
    ('scaling', StandardScaler()),
    ('regression', Ridge(best_alpha))
])

model = simple_pipeline.fit(X_train[numeric_features], y_train)
```

```
y_pred = model.predict(X_test[numeric_features])
print("Test RMSE = %.4f" % mean_squared_error(y_test, y_pred, squared=False))
```

Test RMSE = 31703.3590

Часть 3. Работаем с категориальными признаками

Сейчас мы явно вытягиваем из данных не всю информацию, что у нас есть, просто потому, что мы не используем часть признаков. Эти признаки в датасете закодированы строками, каждый из них обозначает некоторую категорию. Такие признаки называются категориальными. Давайте выделим такие признаки и сразу заполним пропуски в них специальным значением (то, что у признака пропущено значение, само по себе может быть хорошим признаком).

```
In [22]: categorical = list(X_train.dtypes[X_train.dtypes == "object"].index)
X_train[categorical] = X_train[categorical].fillna("NotGiven")
X_test[categorical] = X_test[categorical].fillna("NotGiven")
```

```
In [23]: X_train[categorical].sample(5)
```

```
Out [23]:
```

	MSZoning	Street	Alley	LotShape	LandContour	Utilities	LotConfig	La
184	RL	Pave	NotGiven	IR1	Lvl	AllPub	Inside	
837	RM	Pave	NotGiven	Reg	Lvl	AllPub	Inside	
1094	RL	Pave	NotGiven	Reg	Lvl	AllPub	Inside	
1291	RM	Pave	NotGiven	Reg	Lvl	AllPub	Inside	
436	RM	Pave	NotGiven	Reg	Lvl	AllPub	Inside	

5 rows x 43 columns

Сейчас нам нужно как-то закодировать эти категориальные признаки числами, ведь линейная модель не может работать с такими абстракциями. Два стандартных трансформера из `sklearn` для работы с категориальными признаками — `OrdinalEncoder` (просто перенумеровывает значения признака натуральными числами) и `OneHotEncoder`.

`OneHotEncoder` ставит в соответствие каждому признаку целый вектор, состоящий из нулей и одной единицы (которая стоит на месте, соответствующем принимаемому значению, таким образом кодируя его).

Вопрос: Проинтерпретируйте, что означают веса модели перед `OneHot`-кодированными признаками. Почему пользоваться `OrdinalEncoder` в случае линейной модели — скорее плохой вариант? Какие недостатки есть у `OneHot`-кодирования?

```
In [24]: from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
```

```

column_transformer = ColumnTransformer([
    ('ohe', OneHotEncoder(handle_unknown="ignore"), categorical),
    ('scaling', StandardScaler(), numeric_features)
])

pipeline = Pipeline(steps=[
    ('ohe_and_scaling', column_transformer),
    ('regression', Ridge())
])

model = pipeline.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Test RMSE = %.4f" % mean_squared_error(y_test, y_pred, squared=False))

```

Test RMSE = 26282.2264

Вопрос: Как вы думаете, почему мы не производим скейлинг OneHot-кодированных признаков?

Посмотрим на размеры матрицы после OneHot-кодирования:

```

In [25]: print("Size before OneHot:", X_train.shape)
        print("Size after OneHot:", column_transformer.transform(X_train).shape)

```

Size before OneHot: (1022, 79)

Size after OneHot: (1022, 294)

Как видим, количество признаков увеличилось более, чем в 3 раза. Это может повысить риски переобучиться: соотношение количества объектов к количеству признаков сильно сократилось.

Попытаемся обучить линейную регрессию с ℓ_1 -регуляризатором. На лекциях вы узнаете, что ℓ_1 -регуляризатор разреживает признаковое пространство, иными словами, такая модель зануляет часть весов.

```

In [26]: from sklearn.linear_model import Lasso

column_transformer = ColumnTransformer([
    ('ohe', OneHotEncoder(handle_unknown="ignore"), categorical),
    ('scaling', StandardScaler(), numeric_features)
])

lasso_pipeline = Pipeline(steps=[
    ('ohe_and_scaling', column_transformer),
    ('regression', Lasso())
])

model = lasso_pipeline.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("RMSE = %.4f" % mean_squared_error(y_test, y_pred, squared=False))

```

RMSE = 26468.4015

```

In [27]: ridge_zeros = np.sum(pipeline.steps[-1][-1].coef_ == 0)
        lasso_zeros = np.sum(lasso_pipeline.steps[-1][-1].coef_ == 0)
        print("Zero weights in Ridge:", ridge_zeros)
        print("Zero weights in Lasso:", lasso_zeros)

```

Zero weights in Ridge: 0
Zero weights in Lasso: 29

Подберем для нашей модели оптимальный коэффициент регуляризации.
Обратите внимание, как перебираются параметры у `Pipeline`.

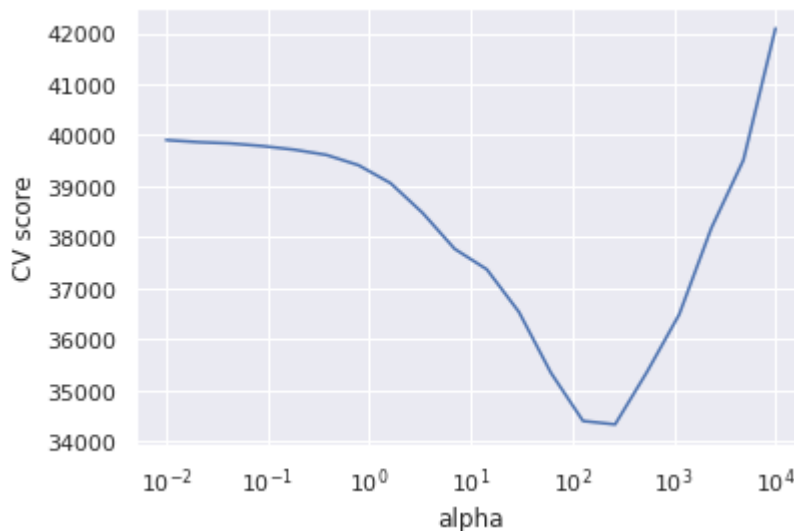
```
In [28]: alphas = np.logspace(-2, 4, 20)
searcher = GridSearchCV(lasso_pipeline, [{"regression__alpha": alphas}],
                        scoring="neg_root_mean_squared_error", cv=10, n_j
searcher.fit(X_train, y_train)

best_alpha = searcher.best_params_["regression__alpha"]
print("Best alpha = %.4f" % best_alpha)

plt.plot(alphas, -searcher.cv_results_["mean_test_score"])
plt.xscale("log")
plt.xlabel("alpha")
plt.ylabel("CV score")
```

Best alpha = 263.6651

Out[28]: Text(0, 0.5, 'CV score')



```
In [29]: column_transformer = ColumnTransformer([
    ('ohe', OneHotEncoder(handle_unknown="ignore"), categorical),
    ('scaling', StandardScaler(), numeric_features)
])

pipeline = Pipeline(steps=[
    ('ohe_and_scaling', column_transformer),
    ('regression', Lasso(best_alpha))
])

model = pipeline.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Test RMSE = %.4f" % mean_squared_error(y_test, y_pred, squared=False))
```

Test RMSE = 25642.6115

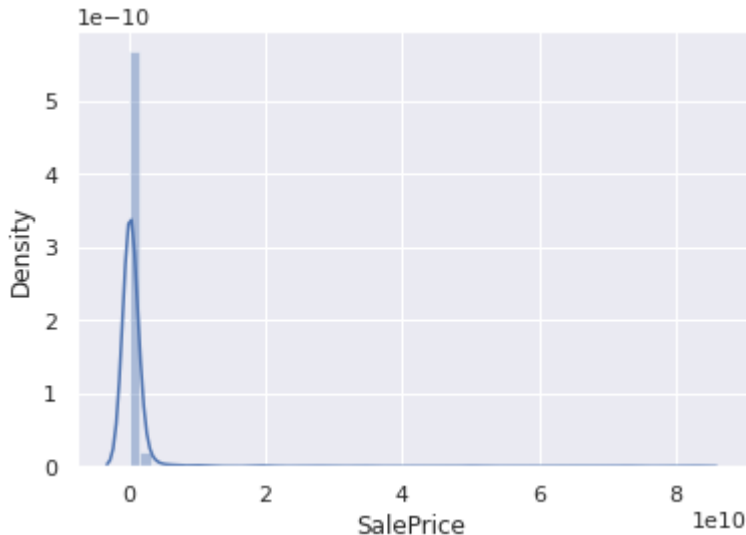
```
In [30]: lasso_zeros = np.sum(pipeline.steps[-1][-1].coef_ == 0)
print("Zero weights in Lasso:", lasso_zeros)
```

Zero weights in Lasso: 218

Иногда очень полезно посмотреть на распределение остатков. Нарисуем гистограмму распределения квадратичной ошибки на обучающих объектах:

```
In [31]: error = (y_train - model.predict(X_train)) ** 2
sns.distplot(error)
```

```
Out[31]: <AxesSubplot:xlabel='SalePrice', ylabel='Density'>
```



Как видно из гистограммы, есть примеры с очень большими остатками. Попробуем их выбросить из обучающей выборки. Например, выбросим примеры, остаток у которых больше 0.95-квантили.

```
In [32]: mask = (error < np.quantile(error, 0.95))
```

```
In [33]: column_transformer = ColumnTransformer([
    ('ohe', OneHotEncoder(handle_unknown="ignore"), categorical),
    ('scaling', StandardScaler(), numeric_features)
])

pipeline = Pipeline(steps=[
    ('ohe_and_scaling', column_transformer),
    ('regression', Lasso(best_alpha))
])

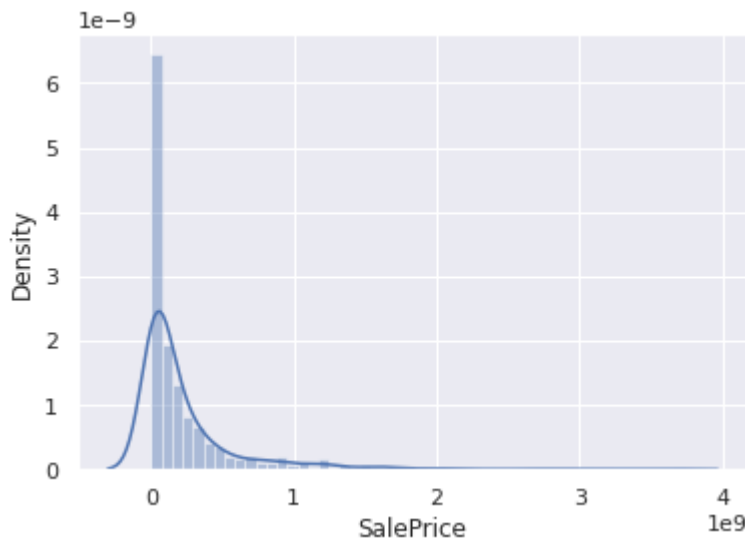
model = pipeline.fit(X_train[mask], y_train[mask])
y_pred = model.predict(X_test)
print("Test RMSE = %.4f" % mean_squared_error(y_test, y_pred, squared=False))
```

Test RMSE = 24963.5922

```
In [34]: X_train = X_train[mask]
y_train = y_train[mask]
```

```
In [35]: error = (y_train[mask] - model.predict(X_train[mask])) ** 2
sns.distplot(error)
```

```
Out[35]: <AxesSubplot:xlabel='SalePrice', ylabel='Density'>
```



Видим, что качество модели заметно улучшилось! Также бывает очень полезно посмотреть на примеры с большими остатками и попытаться понять, почему же модель на них так сильно ошибается: это может дать понимание, как модель можно улучшить.

Часть 4. Подготовка данных для линейных моделей

Есть важное понятие, связанное с применением линейных моделей, — *спрямляющее пространство*. Под ним понимается такое признаковое пространство для наших объектов, в котором линейная модель хорошо описывает данные, даёт хорошее качество прогнозов.

Не существует общих рекомендаций о том, как найти спрямляющее пространство для произвольной выборки. Есть лишь некоторые общие советы — например, если добавить в выборку полиномиальных признаков, то скорее всего модель станет работать лучше (если не переобучится). Пример про этот подход мы обсуждали в начале семинара. Есть и другие трюки.

У линейных моделей есть огромное преимущество: они имеют мало параметров, а поэтому их можно обучить даже на небольшой выборке. Если выборка большая, то параметры модели получится оценить более надёжно — но в то же время есть риск, что данные будут слишком разнообразными, чтобы линейная модель могла уловить все закономерности в них. Иногда можно улучшить ситуацию путём разбиения признакового пространства на несколько областей и построения своей модели в каждой из них.

Попробуем для примера в нашей задаче разделить выборку на две части по признаку OverallQual. Это один из самых сильных признаков, и, возможно, разбиение по нему даст нам две выборки с заведомо разными ценами на дома.

Для начала вспомним, какое качество получается у обычной гребневой регрессии.

```
In [36]: column_transformer = ColumnTransformer([
    ('ohe', OneHotEncoder(handle_unknown="ignore"), categorical),
    ('scaling', StandardScaler(), numeric_features)
])

pipeline = Pipeline(steps=[
    ('ohe_and_scaling', column_transformer),
    ('regression', Ridge())
])

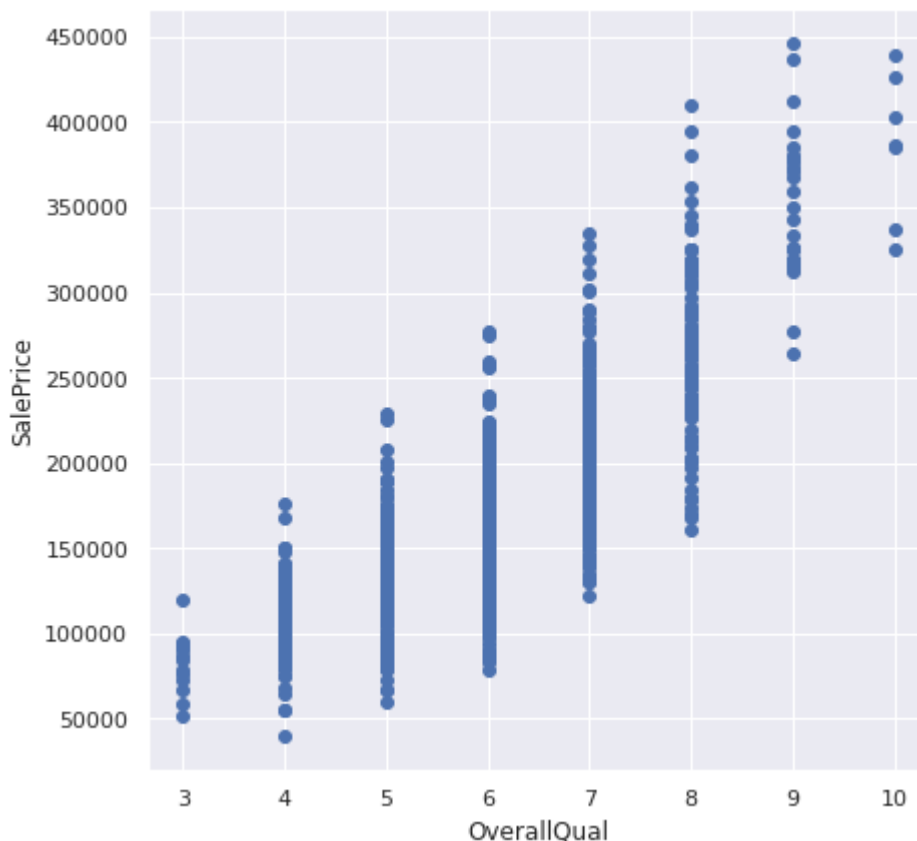
model = pipeline.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Test RMSE = %.4f" % mean_squared_error(y_test, y_pred, squared=False))
```

Test RMSE = 23427.2492

Посмотрим на связь OverallQual и целевой переменной.

```
In [37]: plt.figure(figsize=(7, 7))
plt.scatter(X_train.OverallQual, y_train)
plt.xlabel('OverallQual')
plt.ylabel('SalePrice')
```

Out[37]: Text(0, 0.5, 'SalePrice')



```
In [38]: threshold = 5
mask = (X_train.OverallQual <= threshold)
X_train_1 = X_train[mask]
y_train_1 = y_train[mask]
X_train_2 = X_train[~mask]
y_train_2 = y_train[~mask]
```

```
In [39]: column_transformer1 = ColumnTransformer([
    ('ohe', OneHotEncoder(handle_unknown="ignore"), categorical),
    ('scaling', StandardScaler(), numeric_features)
])

pipeline1 = Pipeline(steps=[
    ('ohe_and_scaling', column_transformer1),
    ('regression', Ridge())
])

column_transformer2 = ColumnTransformer([
    ('ohe', OneHotEncoder(handle_unknown="ignore"), categorical),
    ('scaling', StandardScaler(), numeric_features)
])

pipeline2 = Pipeline(steps=[
    ('ohe_and_scaling', column_transformer2),
    ('regression', Ridge())
])

model1 = pipeline1.fit(X_train_1, y_train_1)
model2 = pipeline2.fit(X_train_2, y_train_2)

y_pred_1 = model1.predict(X_test)
y_pred_2 = model2.predict(X_test)
mask_test = (X_test.OverallQual <= threshold)
y_pred = y_pred_1.copy()
y_pred[~mask_test] = y_pred_2[~mask_test]

print("Test RMSE = %.4f" % mean_squared_error(y_test, y_pred, squared=False))
```

Test RMSE = 22745.7569

Получилось лучше! И это при практически случайном выборе разбиения. Если бы мы поработали над этим получше, то и качество, скорее всего, получилось бы выше.

Перейдём к следующему трюку — бинаризации признаков. Мы выбираем n порогов $t_1, \dots, t_n$ для признака x_j и генерируем $n + 1$ новый признак: $[x_j \leq t_1], [t_1 < x_j \leq t_2], \dots, [t_{n-1} < x_j \leq t_n], [x_j > t_n]$. Такое преобразование может неплохо помочь в случае, если целевая переменная нелинейно зависит от одного из признаков. Рассмотрим синтетический пример.

```
In [40]: from sklearn.preprocessing import KBinsDiscretizer
```

```
In [41]: x_plot = np.linspace(0, 1, 10000)

X = np.random.uniform(0, 1, size=30)
y = np.cos(1.5 * np.pi * X) + np.random.normal(scale=0.1, size=X.shape)

fig, axs = plt.subplots(figsize=(16, 4), ncols=2)

regr = LinearRegression()
regr.fit(X[:, np.newaxis], y)
y_pred_regr = regr.predict(x_plot[:, np.newaxis])
```

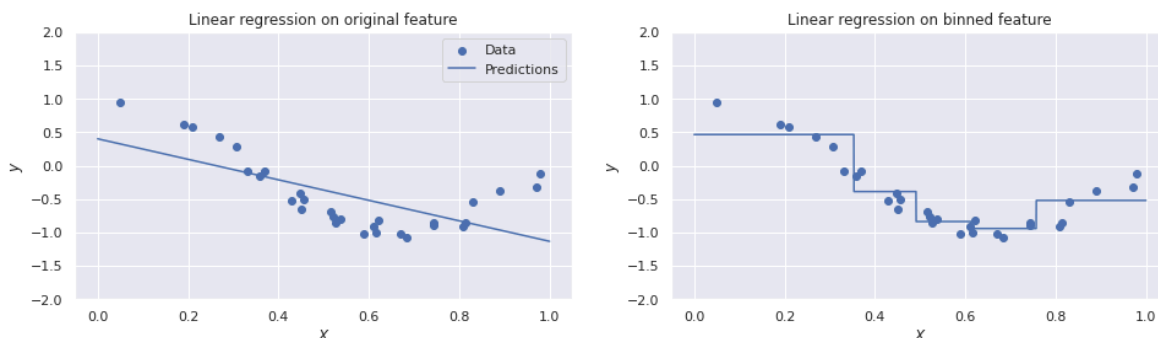
```

axs[0].scatter(X[:, np.newaxis], y, label="Data")
axs[0].plot(x_plot, y_pred_regr, label="Predictions")
axs[0].legend()
axs[0].set_title("Linear regression on original feature")
axs[0].set_xlabel("$X$")
axs[0].set_ylabel("$y$")
axs[0].set_ylim(-2, 2)

binner = KBinsDiscretizer(n_bins=5, strategy='quantile')
pipeline = Pipeline(steps=[
    ('binning', binner),
    ('regression', LinearRegression())
])
pipeline.fit(X[:, np.newaxis], y)
y_pred_binned = pipeline.predict(x_plot[:, np.newaxis])
axs[1].scatter(X[:, np.newaxis], y, label="Data")
axs[1].plot(x_plot, y_pred_binned, label="Predictions")
axs[1].set_title("Linear regression on binned feature")
axs[1].set_xlabel("$X$")
axs[1].set_ylabel("$y$")
axs[1].set_ylim(-2, 2)

```

Out [41]: (-2.0, 2.0)



Видно, что качество модели существенно возросло. С другой стороны, увеличилось и количество параметров модели (из-за увеличения числа признаков), поэтому при бинаризации важно контролировать переобучение.

Иногда может помочь преобразование целевой переменной. Может оказаться, что по мере роста признаков целевая переменная меняется экспоненциально. Например, может оказаться, что при линейном уменьшении продолжительности видео число его просмотров растёт экспоненциально. Учесть это можно с помощью логарифмирования целевой переменной — ниже синтетический пример с такой ситуацией.

```

In [42]: X = np.random.exponential(1, size=30)
y = np.exp(X) + np.random.normal(scale=0.1, size=X.shape)

x_plot = np.linspace(np.min(X), np.max(X), 10000)

fig, axs = plt.subplots(figsize=(16, 4), ncols=2)

regr = LinearRegression()
regr.fit(X[:, np.newaxis], y)
y_pred_regr = regr.predict(x_plot[:, np.newaxis])
axs[0].scatter(X[:, np.newaxis], y, label="Data")

```

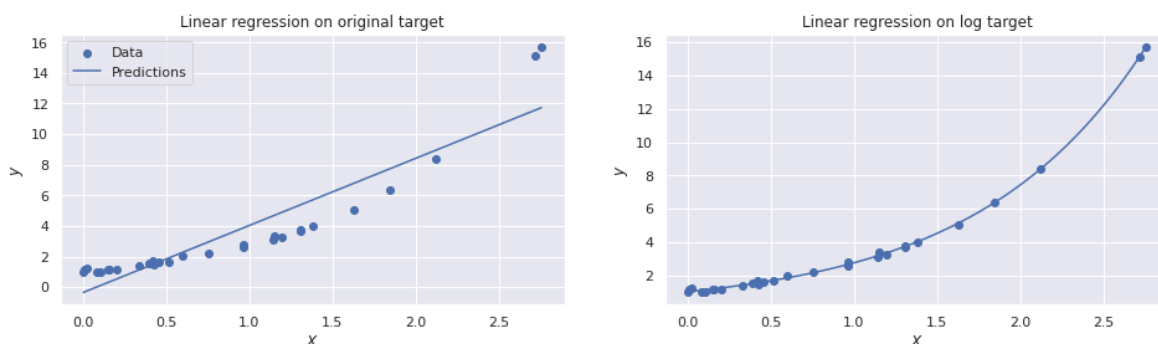
```

axs[0].plot(x_plot, y_pred_regr, label="Predictions")
axs[0].legend()
axs[0].set_title("Linear regression on original target")
axs[0].set_xlabel("$X$")
axs[0].set_ylabel("$y$")

y_log = np.log(y)
regr.fit(X[:, np.newaxis], y_log)
y_pred_log = np.exp(regr.predict(x_plot[:, np.newaxis]))
axs[1].scatter(X[:, np.newaxis], y, label="Data")
axs[1].plot(x_plot, y_pred_log, label="Predictions")
axs[1].set_title("Linear regression on log target")
axs[1].set_xlabel("$X$")
axs[1].set_ylabel("$y$")

```

Out[42]: Text(0, 0.5, '\$y\$')



Но, конечно, вряд ли в реальных данных будет действительно экспоненциальная связь между целевой переменной и линейной комбинацией признаков. Тем не менее, логарифмирование всё равно может помочь.

Ниже пример с данными `boston` по мотивам [примера](#) из документации sklearn. Здесь используется несколько более сложное преобразование: вычисляются квантили распределения целевой переменной и квантили нормального распределения, после чего каждое значение целевой переменной переводится в значение, имеющее такую же квантиль у нормального распределения.

Обратите внимание на класс `TransformedTargetRegressor`, который как раз удобен для построения моделей на преобразованной целевой переменной.

```

In [43]: from sklearn.datasets import load_boston
from sklearn.preprocessing import QuantileTransformer, quantile_transform
from sklearn.linear_model import RidgeCV
from sklearn.compose import TransformedTargetRegressor
from sklearn.metrics import mean_absolute_error, r2_score

```

```

In [44]: dataset = load_boston()
target = np.array(dataset.feature_names) == "DIS"
X = dataset.data[:, np.logical_not(target)]
y = dataset.data[:, target].squeeze()
y_trans = quantile_transform(dataset.data[:, target],
                             n_quantiles=300,
                             output_distribution='normal',
                             copy=True).squeeze()

```

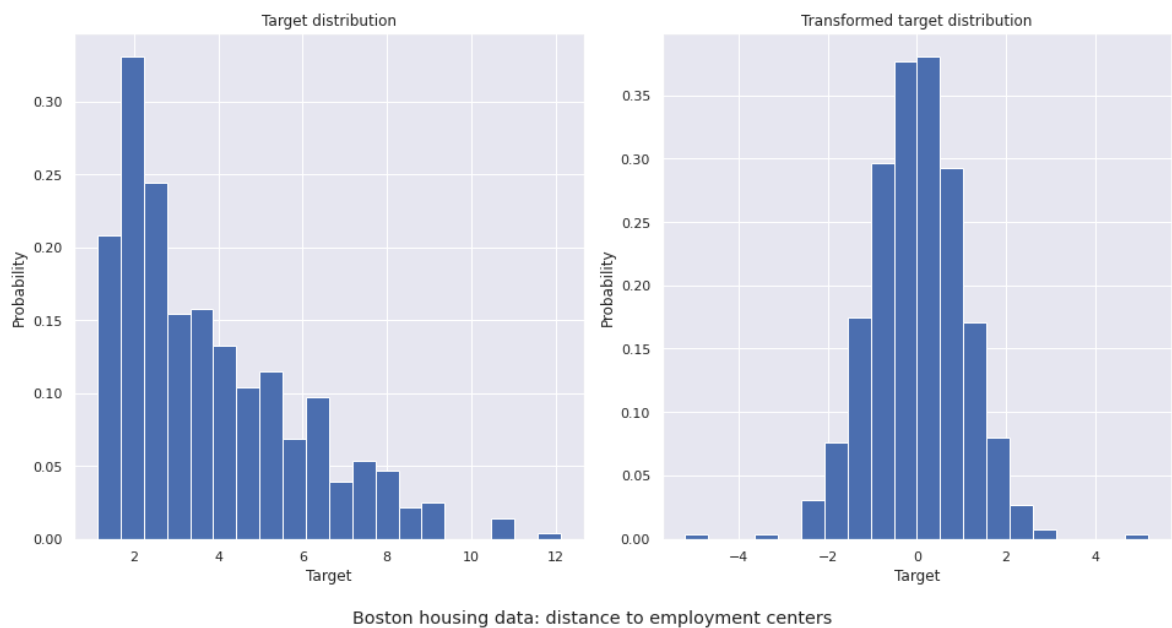
```
In [45]: f, (ax0, ax1) = plt.subplots(1, 2, figsize=(15,8))

ax0.hist(y, bins=20, density=True)
ax0.set_ylabel('Probability')
ax0.set_xlabel('Target')
ax0.set_title('Target distribution')

ax1.hist(y_trans, bins=20, density=True)
ax1.set_ylabel('Probability')
ax1.set_xlabel('Target')
ax1.set_title('Transformed target distribution')

f.suptitle("Boston housing data: distance to employment centers", y=0.035)
f.tight_layout(rect=[0.05, 0.05, 0.95, 0.95])

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=1)
```



```
In [46]: f, (ax0, ax1) = plt.subplots(1, 2, sharey=True, figsize=(15,8))

regr = RidgeCV()
regr.fit(X_train, y_train)
y_pred = regr.predict(X_test)

print("Test MAE without target transformation = %.4f" % mean_absolute_err)
print("Test RMSE without target transformation = %.4f" % mean_squared_err)

ax0.scatter(y_test, y_pred)
ax0.plot([0, 10], [0, 10], '--k')
ax0.set_ylabel('Target predicted')
ax0.set_xlabel('True Target')
ax0.set_title('Ridge regression \n without target transformation')
ax0.set_xlim([0, 10])
ax0.set_ylim([0, 10])

regr_trans = TransformedTargetRegressor(
    regressor=RidgeCV(),
    transformer=QuantileTransformer(n_quantiles=300,
                                     output_distribution='normal'))

regr_trans.fit(X_train, y_train)
y_pred = regr_trans.predict(X_test)
```

```

print("Test MAE with target transformation = %.4f" % mean_absolute_error(
print("Test RMSE with target transformation = %.4f" % mean_squared_error(

ax1.scatter(y_test, y_pred)
ax1.plot([0, 10], [0, 10], '--k')
ax1.set_ylabel('Target predicted')
ax1.set_xlabel('True Target')
ax1.set_title('Ridge regression \n with target transformation')
ax1.set_xlim([0, 10])
ax1.set_ylim([0, 10])

f.suptitle("Boston housing data: distance to employment centers", y=0.035)
f.tight_layout(rect=[0.05, 0.05, 0.95, 0.95])

```

Test MAE without target transformation = 0.8837
 Test RMSE without target transformation = 1.1565
 Test MAE with target transformation = 0.8093
 Test RMSE with target transformation = 1.1548



Видно, что преобразование целевой переменной дало улучшение, хоть и не очень большое. При этом, как видно из графика, улучшения произошли в основном для небольших значений целевой переменной — именно поэтому изменения больше заметны с точки зрения MAE, а не RMSE.

Часть 5. Эконометрический взгляд.

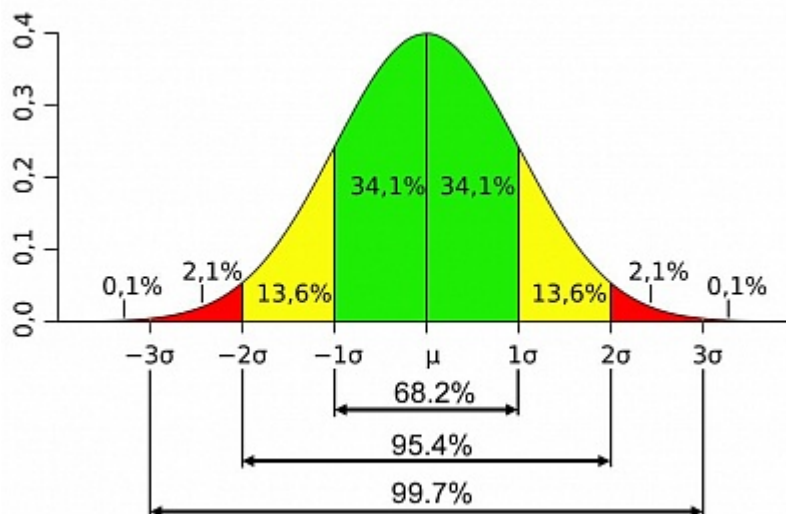
Эконометрика – это раздел прикладной математики, который изучает взаимосвязи между данными. В то время как классический ML заточен на получение лучшей метрики, эконометрика изучает причинно-следственные связи и качественные выводы из данных. В этой науке принята несколько другая терминология, давайте с ней познакомимся!

y – зависимая переменная, x_i – объясняющая переменная/регрессор.

Классическое уравнение линейной регрессии записывают в виде:

$$y = \alpha + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon, \epsilon \sim N(0, \sigma^2)$$

Вопрос: что будет, если $\epsilon \sim N(a, \sigma^2)$?



Мы уже поняли, что признаками могут выступать и степени/кросс-произведения уже использованных признаков (полиномиальная модель), но как можно задать уравнение по-другому? Конкретный вид уравнения линейной регрессии выше называется **спецификацией**.

Спецификации бывают:

1. Квадратичная. $y = \alpha + \beta_1 x + \beta_2 x^2 + \epsilon$. Интерпретация: максимальная значение зависимой переменной достигается при $x_1^* = \frac{-\beta_1}{2\beta_2}$. **Вопрос:** докажите это и приведите примеры, когда зависимость может иметь такой вид.
2. Логарифмическая. $\log y = \alpha + \beta_1 \log x$. Интерпретация: при изменении x на 1% y изменяется на $\beta\%$. **Вопрос:** докажите, что это так через эластичность.
3. Полулогарифмическая в регрессорах. $y = \alpha + \beta \log x$. Интерпретация: при изменении x на 1% y изменяется на $\frac{\beta}{100}$ единиц.

$$\frac{dy}{dx} = \frac{\beta}{x} \Rightarrow \Delta y \approx \beta \frac{\Delta x}{x}$$

4. Полулогарифмическая в зависимой переменной. $\log y = \alpha + \beta x$.
Интерпретация: при изменении x на 1 единицу y изменяется на $e^\beta - 1$ %.

$$5. \quad \frac{\Delta y}{y} = \frac{e^{\alpha + \beta(x + \Delta x)} - e^{\alpha + \beta x}}{e^{\alpha + \beta x}} = e^{\beta \Delta x} - 1$$

Как только мы откалибровали модель с помощью МНК, мы получили функции $\hat{\beta}_i(X, y)$, которые называются **оценками**.

Вопрос: являются ли оценки случайными величинами?

Несмещенность. Оценки называются несмещенными, если $E\hat{\beta}(X, y) = \beta$.

Эффективность. Оценки называются эффективными в некотором классе оценок, если $Var\hat{\beta}(X, y)$ минимальная среди всех оценок данного класса.

Теорема Гауссова-Маркова

Если модель линейной регрессии верно специфицирована и:

1. Ошибки ϵ не коррелируют между наблюдениями, т.е. $cov(\epsilon_i, \epsilon_j) = 0$
2. Дисперсия ошибок ϵ постоянна и не зависит от наблюдения, т.е.

$$Var(\epsilon_i) = \sigma^2$$
3. $E\epsilon = 0$
4. Признаки $x_1, x_2, \dots, x_n$ линейно независимы.

Тогда оценки, полученные с помощью МНК несмещенные и эффективные в классе линейных оценок.

NB! В Теореме Гауссова-Маркова вовсе не требуется нормальность ошибок ϵ .

```
In [2]: # иллюстрация идеи несмещенности
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# параметры "истинной" модели
beta_0_true = 1.0
beta_1_true = 2.0
sigma = 1.0          # стандартное отклонение шума

n = 50                # размер выборки в одном эксперименте
n_sims = 10_000       # число повторов эксперимента

# фиксируем x
x = np.random.uniform(0, 1, size=n)
X = np.column_stack([np.ones(n), x]) # матрица регрессоров (столбец един

beta_0_hats = np.empty(n_sims)
beta_1_hats = np.empty(n_sims)

for s in range(n_sims):
    eps = np.random.normal(0, sigma, size=n)
    y = beta_0_true + beta_1_true * x + eps # генерируем отклик

    # Оценка МНК: beta_hat = (X'X)^(-1) X'y
    beta_hat = np.linalg.lstsq(X, y, rcond=None)[0]

    beta_0_hats[s] = beta_hat[0]
    beta_1_hats[s] = beta_hat[1]

print("Истинные значения:")
print("beta_0 =", beta_0_true, ", beta_1 =", beta_1_true)
print()
print("Средние оценки по", n_sims, "симуляциям:")
print("E[beta_0_hat] ≈", beta_0_hats.mean())
print("E[beta_1_hat] ≈", beta_1_hats.mean())
```

```

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].hist(beta_0_hats, bins=40, density=True)
axes[0].axvline(beta_0_true, linestyle="--", linewidth=2, label="истинное")
axes[0].axvline(beta_0_hats.mean(), linestyle="-. ", linewidth=2, label="среднее")
axes[0].set_title("Оценки beta_0")
axes[0].legend()

axes[1].hist(beta_1_hats, bins=40, density=True)
axes[1].axvline(beta_1_true, linestyle="--", linewidth=2, label="истинное")
axes[1].axvline(beta_1_hats.mean(), linestyle="-. ", linewidth=2, label="среднее")
axes[1].set_title("Оценки beta_1")
axes[1].legend()

plt.tight_layout()
plt.show()

```

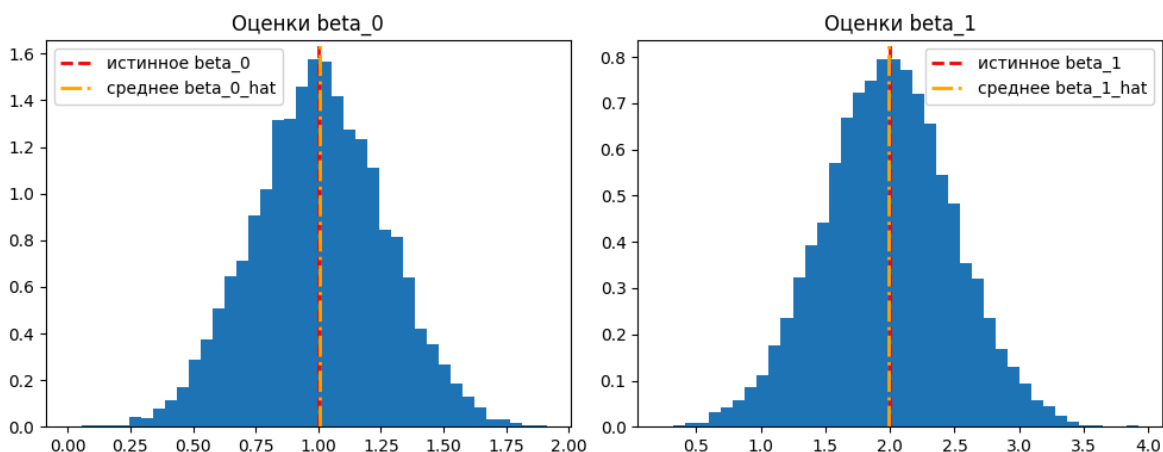
Истинные значения:

$\beta_0 = 1.0$, $\beta_1 = 2.0$

Средние оценки по 10000 симуляциям:

$E[\beta_0_{\text{hat}}] \approx 1.0053443563899476$

$E[\beta_1_{\text{hat}}] \approx 1.98544141568556$



```

In [4]: # иллюстрация идеи несмещенности, теперь изменим мат. ожидание ошибок
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# параметры "истинной" модели
beta_0_true = 1.0
beta_1_true = 2.0
sigma = 1.0          # стандартное отклонение шума

n = 50                # размер выборки в одном эксперименте
n_sims = 10_000       # число повторов эксперимента

# фиксируем x
x = np.random.uniform(0, 1, size=n)
X = np.column_stack([np.ones(n), x]) # матрица регрессоров (столбец един

beta_0_hats = np.empty(n_sims)
beta_1_hats = np.empty(n_sims)

for s in range(n_sims):
    eps = np.random.normal(2, sigma, size=n)

```

```

y = beta_0_true + beta_1_true * x + eps # генерируем отклик

# Оценка МНК: beta_hat = (X'X)^(-1) X'y
beta_hat = np.linalg.lstsq(X, y, rcond=None)[0]

beta_0_hats[s] = beta_hat[0]
beta_1_hats[s] = beta_hat[1]

print("Истинные значения:")
print("beta_0 =", beta_0_true, ", beta_1 =", beta_1_true)
print()
print("Средние оценки по", n_sims, "симуляциям:")
print("E[beta_0_hat] ≈", beta_0_hats.mean())
print("E[beta_1_hat] ≈", beta_1_hats.mean())

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].hist(beta_0_hats, bins=40, density=True)
axes[0].axvline(beta_0_true, linestyle="--", linewidth=2, label="истинное")
axes[0].axvline(beta_0_hats.mean(), linestyle="-. ", linewidth=2, label="среднее")
axes[0].set_title("Оценки beta_0")
axes[0].legend()

axes[1].hist(beta_1_hats, bins=40, density=True)
axes[1].axvline(beta_1_true, linestyle="--", linewidth=2, label="истинное")
axes[1].axvline(beta_1_hats.mean(), linestyle="-. ", linewidth=2, label="среднее")
axes[1].set_title("Оценки beta_1")
axes[1].legend()

plt.tight_layout()
plt.show()

```

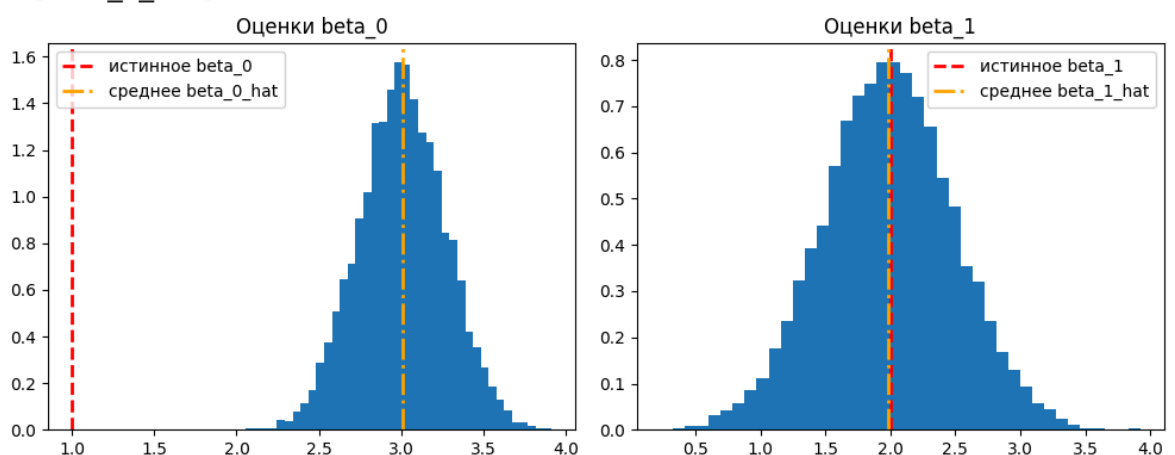
Истинные значения:

beta_0 = 1.0 , beta_1 = 2.0

Средние оценки по 10000 симуляциям:

E[beta_0_hat] ≈ 3.0053443563899473

E[beta_1_hat] ≈ 1.98544141568556



```

In [6]: import numpy as np
import matplotlib.pyplot as plt

# Сравнение эффективности двух оценок beta_1:
# 1) МНК по всей выборке (эффективная)
# 2) МНК только по половине выборки (тоже несмещённая, но менее эффективна)

np.random.seed(42)

```

```

# параметры "истинной" модели
beta_0_true = 1.0
beta_1_true = 2.0
sigma = 1.0          # стандартное отклонение шума

n = 50                # размер выборки в одном эксперименте
n_sims = 10_000       # число симуляций

# фиксируем x (как в теории Гаусса-Маркова)
x = np.random.uniform(0, 1, size=n)
X_full = np.column_stack([np.ones(n), x])

# "неэффективная" оценка
n_half = n // 2
x_half = x[:n_half]
X_half = np.column_stack([np.ones(n_half), x_half])

beta1_full = np.empty(n_sims) # оценка по всей выборке
beta1_half = np.empty(n_sims) # оценка по половине выборки

for s in range(n_sims):
    eps = np.random.normal(0, sigma, size=n)
    y = beta_0_true + beta_1_true * x + eps

    # МНК по всей выборке
    beta_hat_full = np.linalg.lstsq(X_full, y, rcond=None)[0]
    # МНК по половине выборке
    beta_hat_half = np.linalg.lstsq(X_half, y[:n_half], rcond=None)[0]

    beta1_full[s] = beta_hat_full[1]
    beta1_half[s] = beta_hat_half[1]

print("Истинный параметр:")
print("beta_1 =", beta_1_true)
print()
print("Средние оценки по", n_sims, "симуляциям:")
print("E[beta1_full] ≈", beta1_full.mean())
print("E[beta1_half] ≈", beta1_half.mean())
print()
print("Эмпирическая дисперсия оценок:")
print("Var(beta1_full) ≈", beta1_full.var(ddof=1))
print("Var(beta1_half) ≈", beta1_half.var(ddof=1))

fig, axes = plt.subplots(1, 2, figsize=(10, 4), sharey=True)

axes[0].hist(beta1_full, bins=40, density=True)
axes[0].axvline(beta_1_true, linestyle="--", linewidth=2, label="истинный")
axes[0].set_title("beta1_hat (вся выборка)")
axes[0].legend()

axes[1].hist(beta1_half, bins=40, density=True)
axes[1].axvline(beta_1_true, linestyle="--", linewidth=2, label="истинный")
axes[1].set_title("beta1_hat (половина выборки)")
axes[1].legend()

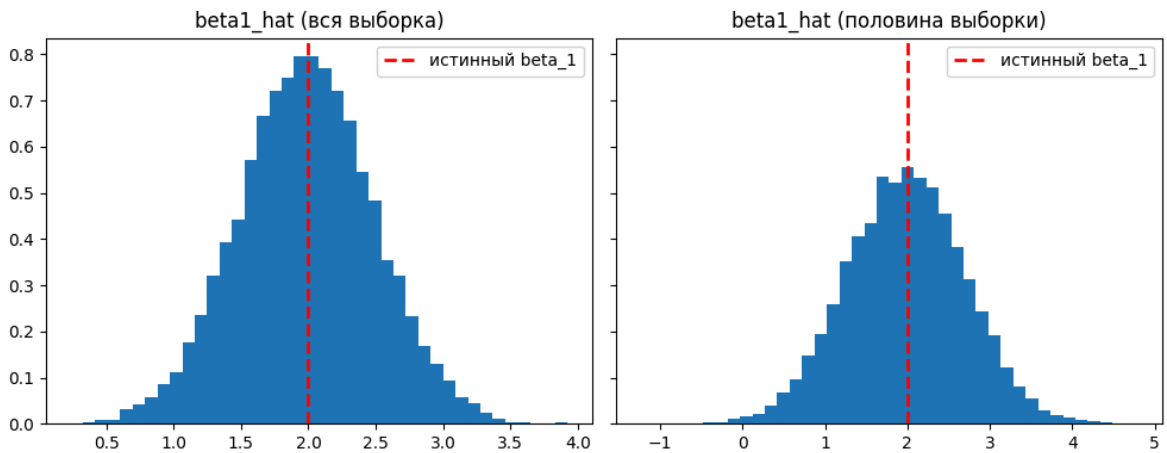
plt.tight_layout()
plt.show()

```

Истинный параметр:
 $\beta_1 = 2.0$

Средние оценки по 10000 симуляциям:
 $E[\beta_{1_full}] \approx 1.98544141568556$
 $E[\beta_{1_half}] \approx 1.9797799211734661$

Эмпирическая дисперсия оценок:
 $\text{Var}(\beta_{1_full}) \approx 0.24990601467914675$
 $\text{Var}(\beta_{1_half}) \approx 0.518692418480375$



Вопрос: В эксперименте мы считали оценку только по половине выборки, что достаточно очевидно влечет большую дисперсию оценки. Однако, когда это имеет смысл?

Ответ:

Когда в выборке присутствуют выбросы и в целом она достаточно гетерогенна. Тогда разбиение на несколько бинов и подсчёт оценки для них, а затем усреднение - помогает быть оценки более эффективной.

In []: